**Task 11. Trouver un motif dans une image** `solve( imageWidth, imageHeight, image, patternWidth, patternHeight, pattern)`

11. **(C++, 40min)** Поиск деталей в изображении/Поиск узора в изображении/Поиск закономерности в изображении. *Но если честно, я так и не понял истинной сути задачи.*

*Règles* On vous donne 2 images : une image de base, et un motif qui peut être présent dans cette image.

Chaque image est représentée sous la form d'un vecteur de chaîne de caractères, òu chaque élément represent une ligne de pixels de l'image, et chaque caractère represent un pixel. Il n'est pas nécessaire de comprendre en detail cet encodage pour résoudre ce problème, mais pour information, des explications sont fournies plus bas. [(ru) Каждое изображение представлено в виде вектора строк, где каждый элемент представляет собой строку пикселей изображения, а каждый символ - пиксель. Для решения этой задачи нет необходимости подробно разбираться в этом кодировании, но для справки ниже приведены пояснения.]

Vous devez retourner la position `x`, `y` de ce motif dans l'image, ou `[-1 ,-1]` si le motif n'est pas présent dans l'image. Si le motif apparaît plusieurs fois dans l'image, retournez la position du plus haut (le `y`plus petit), et en cas d'égalité, le plus à gauche ( le `x` le plus petit). [(ru) Вы должны вернуть `x`, `y` позиции этого паттерна на изображении, или `[-1 ,-1]`, если паттерн отсутствует на изображении. Если деталь встречается на изображении несколько раз, верните самую высокую позицию (наименьший `y`), а в случае равенства - самую левую (наименьший `x`).]

La position du motif est déterminée par les coordonnées `x`, `y` de son coin en haut à gauche. `x`représente la colone, `y` représente la ligne, et les coordonnées [0,0] représente le coin en haut à gauche. [(ru) Положение детали определяется координатами `x`, `y` ее левого верхнего угла. `x` представляет столбец, `y` - строку, а координаты [0,0] - левый верхний угол.]

**Implémentation** Implément la fonction `solve( imageWidth, imageHeight, image, patternWidth, patternHeight, pattern)`où les parameters sont:

- `imageWidth`: `int` la largeur de l'image
- `imageHeight`: `int` la hauteur de l'image
- `image`: `std::vector<std::string>` l'image, un vector de string, où chaque caractère represent un pixel
- `patternWidth`: `int` la largeur du motif
- `patternHeigth`: `int` la hauteur du motif
- `pattern`: `std::vector<std::string>` la largeur de l'image

et qui doit retourner:

- si le motif est présent dans l'image: la position `x`, `y`, sous la form d'un vecteur de 2 entiers, représentant le coin en haut à gauche du 1er motif, en partant du haut vers le bas, et de gauche à droite. [(ru) Если деталь присутствует на изображении: положение `x`, `y` в виде вектора из 2 целых чисел, представляющих верхний левый угол 1-й детали, начиная сверху вниз и слева направо.]
- si le motif n'est pas present dans l'image: [-1,-1].[(ru) если паттерн отсутствует на изображении: [-1,-1]]

*Conditions de Victoire*:

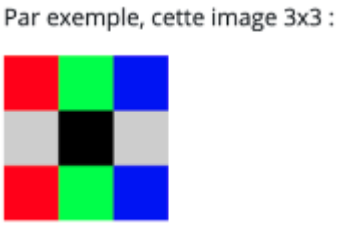
- Le motif est effectivement situé aux coordonnées `x`, `y`.[(ru) Деталь фактически расположена в координатах `x`, `y`.]
- Si le motif apparait plusieurs fois dans l'image retournez la position du plus haut (le `y`plus petit), et en cas d'égalité, le plus à gauche ( le `x` le plus petit). [(ru) Если деталь встречается на изображении несколько раз, верните позицию самого высокого (наименьшего `y'), а в случае равенства – самого левого (наименьшего` x').]

*Conditions de Défaite*

- La sous-image qui commence aux coordonnées `x`, `y` ne correspond pas au motif.[(ru) Под картинка начинающееся в координатах `x`, `y`, не соответствует шаблону.]
- Vous prenez plus d'une seconde à répondre.
- Il y a un autre motif qui correspond, plus haut ou plus à gauche que le votre.[(ru) Есть еще один подходящий узор, расположенный выше или левее вашего.]

*Contraintes*. 1 <= `imageWidth` <= 400 1 <= `imageHeight` <= 400 1 <= `patternWidth` <= 400 1 <= `patternHeight` <= 400 Temp de réponse <= 1s.

*Details sur l'encodage* Initialement, une image est un tableau d'entiers à 2 dimensions, où chaque entier représente une pixel (un utilisant la notation RGB). Toutes nous images possèdent au maximum 62 couleurs différentes, de manière à ce que nous puissions encoder chaque entier en un caractère parmi `0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ`. Nous avons simplement transformé l'image en un tableau de strings, où la 1ère string est la 1ère ligne de l'image, etc.




Par example, cette image 3x3 :

est représentée en tableau d'entiers à 2 dimensions comme:

```
[
  [0xff0000, 0x00ff00, 0x0000ff],
  [0xd3d3d3, 0x000000, 0xd3d3d3],
  [0xff0000, 0x00ff00, 0x0000ff]
]
```

qui est ensuite encodée en un tableau de string en :

```
[
  "012",
  "343",
  "012"
]
```

```cpp
vector<int> solve( int imageWidth, int imageHeight, vector<int>image,
                   int patternWidth, int patternHeight, vector<int>pattern)
{
  return vector<int>();
}
```

Tests:

1. 4 pixels
2. Flag
3. Mario